

QueryQuest: Assignment 2

Vibha Narayan

June 30, 2026

Part 1: Short Answer Questions

Q1

- SQL would return the output as 0 even though the student's entry with *Phone number* = *NULL* exists as it evaluates to UNKNOWN rather than TRUE or FALSE (NULL represents "unknown," not a value) because the WHERE clause only keeps rows where the condition is TRUE, so rows with UNKNOWN get filtered out just like FALSE rows, and the count is 0.

Correct Query: `SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;`

As for why NULL is excluded more generally: NULL doesn't mean "empty string" or "zero" — it means the value isn't known or doesn't apply. Since it's not an actual data value, it can't meaningfully participate in arithmetic or logical comparisons. If you don't know a value, you can't say it equals, is less than, or is greater than anything else — that's exactly why such comparisons return UNKNOWN. This propagates into aggregates: SUM, AVG, MIN, MAX, and COUNT(column) all skip NULLs because including an "unknown" in a sum or average would make the result itself partly unknown/meaningless — there's no sensible numeric contribution an unknown value can make.

Q2

- The student's conclusion is correct (DISTINCT changes nothing) but their reasoning should clarify why — not because DISTINCT "doesn't apply" to GROUP BY queries in general, but because in this specific query the GROUP BY already produces one row per department, making post-aggregation duplicates impossible.

Q3

- INNER JOIN only returns rows where a match exists in both tables — a customer row is included only if it has at least one corresponding order. LEFT JOIN (with Customer as the left table) returns every row from Customer regardless of whether a match exists, filling in NULLs for the Order columns when there's no match.

So if LEFT JOIN returns more rows than INNER JOIN, that tells you, without looking at the tables directly, that there exist customers with no matching orders at all (zero orders placed). The size of the difference actually tells you something more precise too: the difference in row count equals the number of customers who have zero orders — because each such customer contributes exactly one row to the LEFT JOIN result (with NULLs for the Order side) but contributes zero rows to the INNER JOIN result.

Scenario that would cause LEFT JOIN to return more rows: a Customer table containing newly registered customers, or customers who created an account but never completed a purchase. For example, someone signs up on an e-commerce site, browses around, but never places an order — their customer record exists, but there's no corresponding row in Order. Under INNER JOIN, this customer

would simply vanish from the result; under LEFT JOIN, they'd appear once with NULL values in all the Order-derived columns (order_id, order_date, amount, etc.).

Q4

- The conceptual problem: GROUP BY department collapses all the rows for, say, the "Engineering" department into a single output row. AVG(salary) makes sense in that single row because averaging is a well-defined operation that reduces many salary values down to one number. But employee_name has no such reduction — if Engineering has 50 employees, which one of those 50 names should appear in that single collapsed row? There's no principled answer. SQL doesn't pick the first one, the last one, a random one, or anything else — it simply refuses to run the query, because allowing it would mean displaying an arbitrary, non-deterministic, and essentially meaningless value for a column that's supposed to represent something specific about a single employee, when the row itself represents an entire group of employees.

If SQL did "allow" this, the engine would have to silently choose some employee_name from each group, with no guarantee of which one, and that choice could even differ across runs of the same query or across different database engines. This would make query results unreliable and the data itself misleading, since the displayed name wouldn't correspond to anything meaningful — it wouldn't be "the average employee" or "a representative employee," just whichever row the engine happened to process first or last.

Q5

- The error is that WHERE cannot reference an aggregate function like AVG(salary) in this context, because of SQL's logical query execution order. SQL doesn't execute clauses in the order they're written. The conceptual execution order is roughly: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY

WHERE is evaluated before GROUP BY even happens. At the point WHERE runs, the database is still looking at individual Employee rows one at a time — it hasn't grouped anything by department yet, so there's no such thing as "the average salary" available to check against. AVG(salary) only becomes meaningful after grouping has occurred, because an aggregate is a value computed across a set of rows within a group, and WHERE operates on one row at a time before those groups even exist. Asking WHERE to filter on AVG(salary) is asking it to use a value that literally hasn't been computed yet at that stage of execution.

This is exactly why SQL has a separate clause, HAVING, which runs after GROUP BY (and conceptually after the aggregates for each group have been computed). HAVING is built specifically to filter groups based on aggregate conditions, the same way WHERE filters individual rows based on row-level conditions.

Correct Query: SELECT department, AVG(salary) FROM Employee GROUP BY department HAVING AVG(salary) > 80000;

Q6

(a) Find all products priced above the overall average product price

- This is a *non-correlated subquery* situation. The "overall average" is a single fixed number — it doesn't depend on which product row you're currently looking at. So you compute it once, get that one scalar value, and then compare every product's price against that constant.

Query: SELECT name FROM Product WHERE price > (SELECT AVG(price) FROM Product);

(b) Find all employees who earn more than the average salary of their own department

- This is a *correlated subquery* situation. The phrase "their own department" is the giveaway. The

average you're comparing against is different for each employee — an Engineering employee compares against Engineering's average, an HR employee compares against HR's average, and so on. The average you need depends on which employee row you're currently evaluating, so the subquery must reference the outer row.

Query:

```
SELECT name FROM Employee e1
WHERE salary > (
  SELECT AVG(salary) FROM Employee e2
  WHERE e2.department = e1.department
);
```

A non-correlated subquery's cost is essentially constant regardless of how large the outer table grows — you pay the subquery cost once and reuse the result. A correlated subquery's total cost grows linearly with the number of outer rows

Part 2: Long Answer Question

(a) Customers that never placed an order

• **Query:**

```
SELECT * FROM Customer as c
LEFT JOIN "Order" as o
ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

LEFT JOIN is used instead of INNER JOIN because INNER JOIN only returns rows with matches — it would only show customers with orders, which is the opposite of what we want. LEFT JOIN preserves all customer rows regardless of whether a match exists. WHERE o.customer_id IS NULL is the key filter — after the LEFT JOIN, customers with no orders have NULL in all Order-side columns, so this condition precisely identifies the "never ordered" customers. This is the standard anti-join pattern in SQL.

(b) For each restaurant, find the average order value, where an order's value is the sum of price $\times$ quantity across its items. Only include restaurants with more than 5 orders

• **Query:**

```
SELECT restaurant_id, AVG(order_value)
AS avg_order_value
FROM (
  SELECT o.order_id, o.restaurant_id,
  SUM(mi.price * oi.quantity) AS order_value
  FROM "Order" o
  JOIN OrderItem oi ON o.order_id = oi.order_id
  JOIN MenuItem mi ON oi.item_id = mi.item_id
  GROUP BY o.order_id, o.restaurant_id
) AS order_values
GROUP BY restaurant_id
HAVING COUNT(*) > 5;
```

We cannot do this in a single GROUP BY because you need two different levels of aggregation — SUM at the order level, then AVG at the restaurant level. If you tried to flatten it into one query like AVG(SUM(price $\times$ quantity)), SQL doesn't allow nested aggregates. The inner subquery first collapses items into orders, then the outer query collapses orders into restaurants.

Why HAVING and not WHERE for the "more than 5 orders" condition:

COUNT(*) > 5 is a condition on a group (how many orders a restaurant has), not on an individual row, so it must go in HAVING.

- (c) Find restaurants whose average order value is higher than the platform-wide average order value

- **Query:**

```
SELECT restaurant_id, avg_order_value
FROM (
  SELECT restaurant_id, AVG(order_value) AS avg_order_value
  FROM (
    SELECT o.order_id, o.restaurant_id,
    SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
  ) AS order_values
  GROUP BY restaurant_id
  HAVING COUNT(*) > 5
) AS restaurant_avgs
WHERE avg_order_value > (
  SELECT AVG(order_value)
  FROM (
    SELECT o.order_id,
    SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id
  ) AS all_order_values
);
```

The platform-wide average order value is a single fixed number — it does not change depending on which restaurant row is being evaluated. This makes it a *non-correlated subquery* — the inner subquery runs exactly once, produces one scalar value, and the outer WHERE uses that as a fixed threshold for all rows. A correlated subquery would re-run for every restaurant row, which would be wasteful and unnecessary since the result never changes.

Note: The platform-wide average is computed over all orders regardless of the >5 orders filter on restaurants. If the intent is to restrict to only qualifying restaurants, the inner subquery for the platform average could also include the HAVING clause — this is an intentional design choice worth clarifying.

- (d) Find the most ordered menu item (by total quantity) for each restaurant

- **Query:**

```
SELECT restaurant_id, item_id, item_name, total_qty
FROM (
  SELECT mi.restaurant_id, mi.item_id, mi.item_name,
  SUM(oi.quantity) AS total_qty,
  RANK() OVER (PARTITION BY mi.restaurant_id
  ORDER BY SUM(oi.quantity) DESC) AS rnk
  FROM MenuItem mi
  JOIN OrderItem oi ON mi.item_id = oi.item_id
  GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) AS ranked
```

WHERE `rnk = 1`;

The inner subquery first computes the total quantity ordered per item per restaurant using `SUM` and `GROUP BY`, then `RANK()` OVER (`PARTITION BY restaurant_id ORDER BY total_qty DESC`) assigns rank 1 to the highest-quantity item within each restaurant's group. The outer `WHERE rnk = 1` then isolates only the top item per restaurant. A window function is used here instead of a plain `GROUP BY + MAX` because `MAX` can return the maximum *value* per group but cannot directly tell you *which item* achieved that maximum — `item_id` and `item_name` would be neither grouped nor aggregated, causing the same invalidity as Q4.

Limitation of basic GROUP BY + MAX for top-N per group problems:

The fundamental problem is that `MAX` gives you the maximum *value* but not the *row* that produced it. For example, `SELECT restaurant_id, MAX(total_qty) FROM ... GROUP BY restaurant_id` tells you each restaurant's highest quantity, but you cannot include `item_name` or `item_id` in that `SELECT` without either adding them to `GROUP BY` (which would break the grouping) or aggregating them (which is meaningless). Retrieving the full row requires a second correlated subquery or self-join to match the max value back to its item — which becomes verbose and fragile. Worse, if two items tie for the highest quantity in a restaurant, a `MAX`-based approach silently returns only one value with no way to recover both tied items, whereas `RANK()` naturally assigns both rank 1 and surfaces them correctly. This is why window functions like `RANK()` or `ROW_NUMBER()` exist — they are purpose-built for top-N per group retrieval.

Part III: Normalization

Q1: Functional Dependencies

- `student_id` → `student_name`
- `course_id` → `course_name`, `instructor_id`
- `instructor_id` → `instructor_name`, `instructor_office`
- (`student_id`, `course_id`) → `grade`, `enrollment_date`

Q2: Is the table in 1NF?

- Yes. Every attribute holds a single atomic value — there are no repeating groups or multi-valued columns. Each row is uniquely identifiable by the composite key (`student_id`, `course_id`). 1NF is satisfied.

Q3: Partial Dependency violating 2NF

- A partial dependency exists when a non-key attribute depends on only part of the composite key. Here, `student_name` depends solely on `student_id`, and `course_name`, `instructor_id` depend solely on `course_id` — neither requires the full (`student_id`, `course_id`) key.

Update anomaly: If a student changes their name, every row containing that `student_id` must be updated — one per course enrolled. Missing even one row creates inconsistency, with two different names for the same student in the database.

Insertion anomaly: A new course cannot be recorded until at least one student enrolls, because `course_id` alone cannot form a valid row without a `student_id`.

Deletion anomaly: If the only student enrolled in a course drops it, deleting that row destroys all knowledge of that course's name and instructor.

Q4: Transitive Dependency violating 3NF

- Even after resolving partial dependencies, a transitive dependency remains:
`course_id → instructor_id → instructor_name, instructor_office`

`instructor_name` and `instructor_office` depend on `instructor_id`, which is itself a non-key attribute. So these attributes reach the key only indirectly — through another non-key attribute — violating 3NF.

Anomaly: If an instructor moves offices, every course row taught by that instructor must be updated. With 10 courses, that's 10 updates — inconsistency arises if even one is missed. If no courses are currently assigned to an instructor, their office information is lost entirely.

Q5: Decomposition into 3NF

- **Student**(student_id, student_name)
 Justified by: `student_id → student_name` (partial dependency removed)
- **Course**(course_id, course_name, instructor_id)
 Justified by: `course_id → course_name, instructor_id` (partial dependency removed)
- **Instructor**(instructor_id, instructor_name, instructor_office)
 Justified by: `instructor_id → instructor_name, instructor_office` (transitive dependency removed)
- **Enrollment**(student_id, course_id, grade, enrollment_date)
 Justified by: `(student_id, course_id) → grade, enrollment_date` (only full-key dependencies remain)

Every table now satisfies 3NF — each non-key attribute depends on the whole key and nothing but the key. No partial or transitive dependencies remain.

Part IV: Design Justification

- **JOIN vs Subquery (Part II, Q(a)):**
 For finding customers who never placed an order, the query could have been written using either a LEFT JOIN with a NULL check or a subquery using NOT IN / NOT EXISTS. I chose the LEFT JOIN approach because it is more readable and its intent is immediately clear — preserving all customer rows and filtering on the NULL gap makes the anti-join pattern visually obvious. A NOT IN subquery like `WHERE customer_id NOT IN (SELECT customer_id FROM "Order")` achieves the same result but has a subtle correctness trap: if any `customer_id` in the Order table is NULL, NOT IN silently returns no rows at all, because NULL comparisons propagate UNKNOWN. The LEFT JOIN approach handles NULLs correctly by construction, making it both safer and more transparent for this use case.
- **Normalization vs Query Convenience (Part III):**
 Decomposing Enrollment into Student, Course, Instructor, and Enrollment eliminates redundancy and prevents update, insertion, and deletion anomalies. However, this comes at a cost — any query that previously needed only one table now requires multiple JOINS. For example, retrieving a student's name alongside their course name, instructor name, and grade requires joining all four tables, whereas the original denormalized table answered this with a simple SELECT. Real systems sometimes intentionally retain some denormalization — for instance, storing `instructor_name` directly in a course table — when read performance is critical and updates to that field are rare. This trade-off between write correctness (favoring normalization) and read simplicity (favoring denormalization) is a core design tension in database systems, often resolved through careful profiling of actual query workloads.